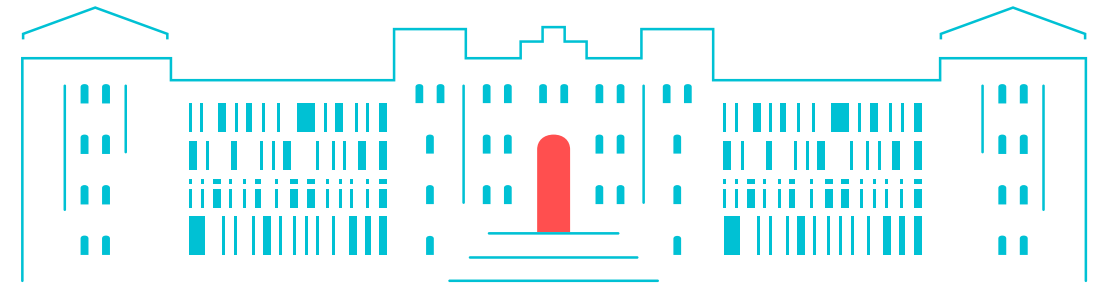
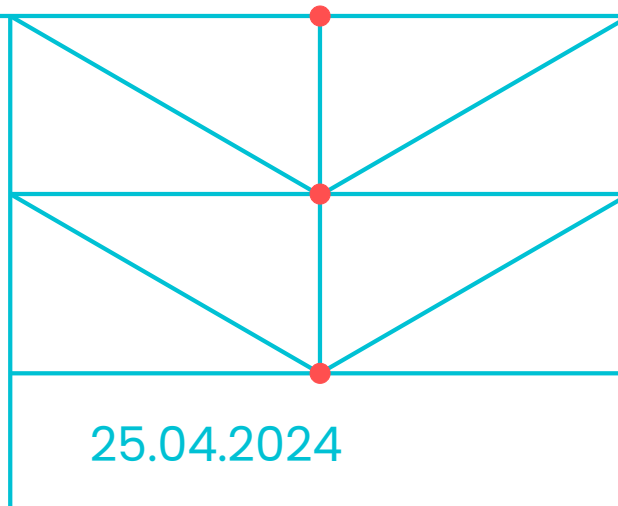


Parallelization of Solvers

Fynn Bense

Chair Computational Mathematics
Institute of Mathematics (E-10)
Building E, Room 3.045

TUHH
Hamburg
University of
Technology

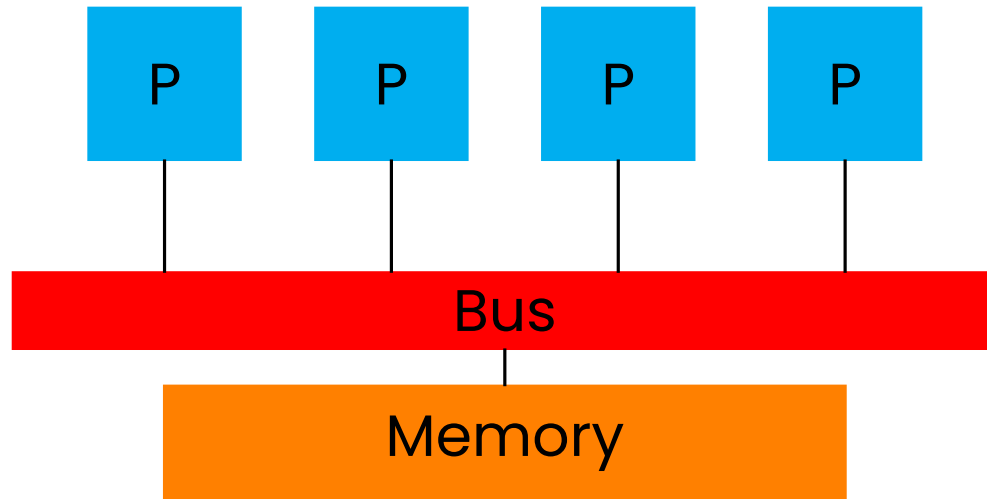


04 – MPI

- Basic MPI vocabulary
- Point-to-point communication
- Deadlocks
- Collective communication

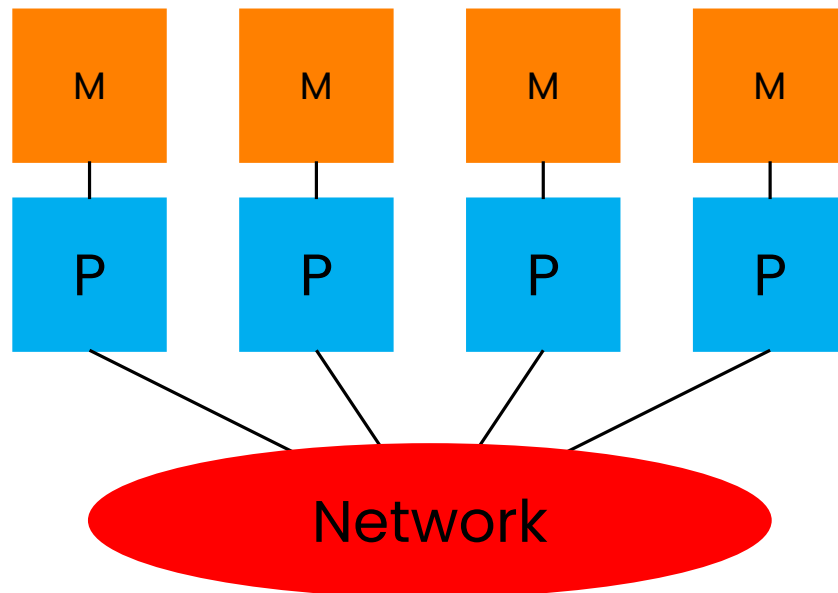
Additional reading

- Official standard: <https://www.mpi-forum.org/>



Shared memory

- Single address space
- All processes have access to the pool of shared memory



Distributed memory

- Each processor has its own local memory
- Message-passing is used to exchange data between processors

GNU C Compiler OpenMP

```
mpicc ...
```

C runtime library definition

```
#include <mpi.h>
```

Program startup

```
mpirun -n <numprocs> <program>
```

```
mpiexec -n <numprocs> <program>
```

GNU C Compiler OpenMP

```
mpicc ...
```

C runtime library definition

```
#include <mpi.h>
```

Program startup

```
mpirun -n <numprocs> <program>
```

```
mpiexec -n <numprocs> <program>
```

```
int MPI_Init(int *argc, char **argv)
```

Initialize MPI library

```
int MPI_Finalize(void)
```

Finalize MPI library

Communicators

```
MPI_Comm MPI_COMM_WORLD;
```

After `MPI_Init` has been called, `MPI_Comm MPI_COMM_WORLD` is a valid handle to predefined communicator that includes all processes available

Size

```
int MPI_Comm_size(MPI_Comm comm, int *size);
```

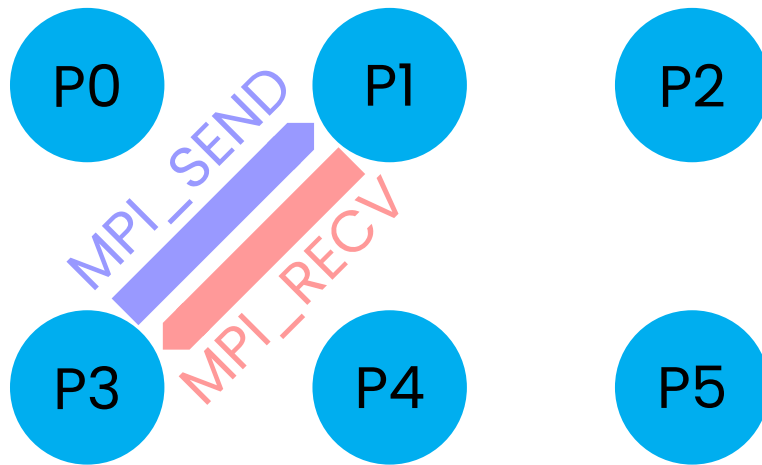
Determine the total number of processes in a communicator

Rank

```
int MPI_Comm_rank(MPI_Comm comm, int *rank);
```

Determine the rank of the calling processes within a communicator

```
1 // This needs to be included to make the MPI library available in your program
2 #include <mpi.h>
3
4 // This we only need to write output
5 #include <stdio.h>
6
7 int main(int argc, char** argv) {
8
9     // Integer to store the number of MPI processes launched
10    int world_size;
11
12    // The rank ("number") of the current process: from 0 to world_size-1
13    int world_rank;
14
15    // Initialize the MPI environment: we need no input parameters, hence NULL
16    MPI_Init(NULL, NULL);
17
18    // MPI_Comm_Size is an MPI command that determines the number of processes. This number
19    // is set by the -n flag in mpirun. This number is the same for all processes.
20    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
21
22    // Get the rank of the process: note that every process will be running a copy of this program
23    // so in every process, world_rank will have a different value
24    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
25
26    // Print "Hello World". Note that every process will print this, so you will see world_size many messages in your
    // terminal
27    printf("Hello world says rank %d out of %d processes \n", world_rank, world_size);
28
29    // Finalize the MPI environment.
30    MPI_Finalize();
31 }
```



- PtP transfers one buffer of data from one process to one other process
- Sending process posts an `MPI_SEND` command
- Receiving process posts an `MPI_RECV` command

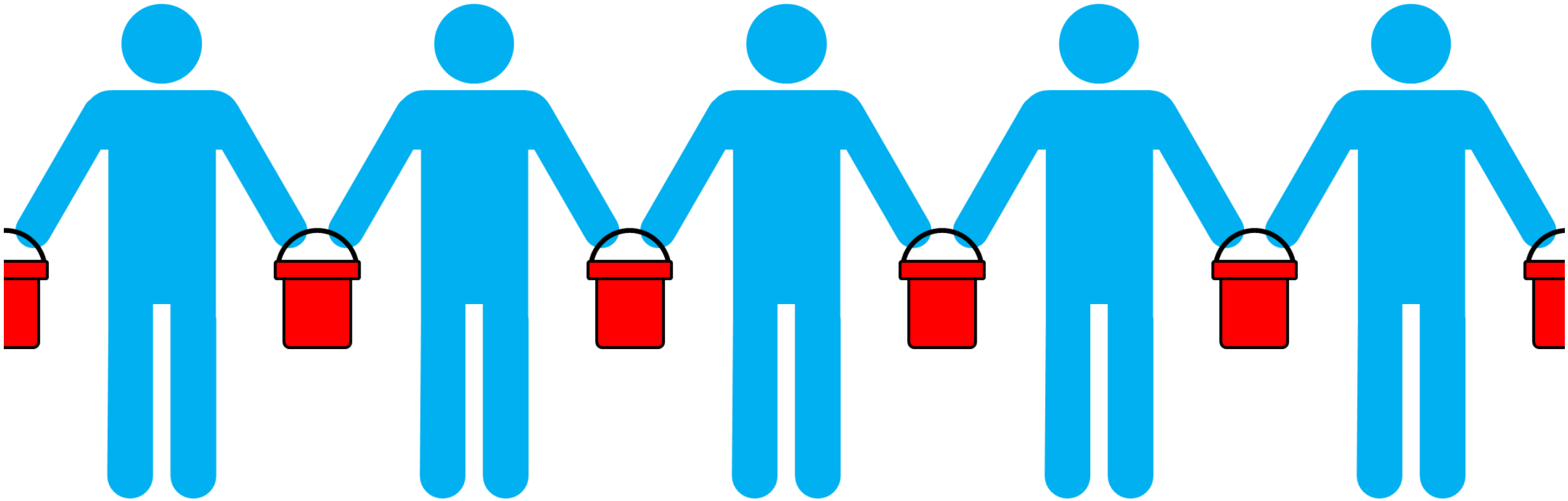
Send

```
int MPI_Send(const void* buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)
```

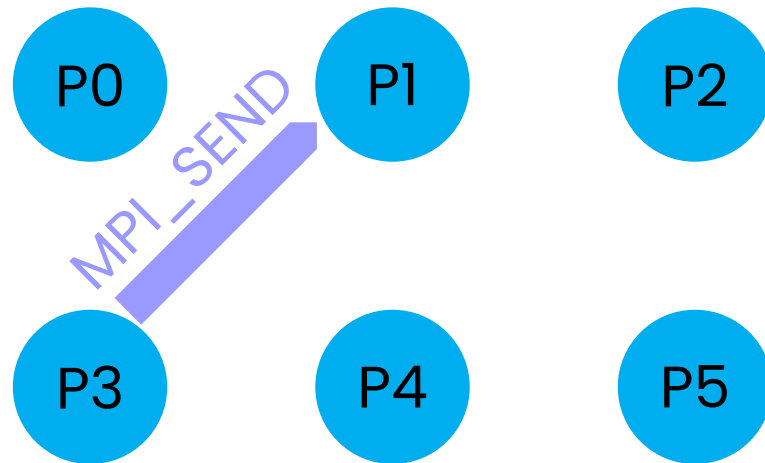
Receive

```
int MPI_Recv(void* buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)
```

- Note that data is only transferred from P3 to P1 in the example above. Transferring data from P1 to P3 would need a second pair of `MPI_SEND` and `MPI_RECV` commands



Have N MPI processes pass a counter from one to the next, each increasing it by one.



- If an `MPI_SEND` is posted without a corresponding `MPI_RECV` or the other way round, the result is a deadlock
- This will result in your program never terminating: the process that posts a send or receive that is not reciprocated will wait forever for the transfer to complete